

An unstructured adaptive mesh refinement approach for computational fluid dynamics of reacting flows

R.S. Cant^{a,*}, U. Ahmed^b, J. Fang^c, N. Chakarborty^b, G. Nivarti^a, C. Moulinec^c, D.R. Emerson^c

^a Cambridge University, Engineering Department, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom

^b School of Engineering, Newcastle University, Claremont Road, Newcastle-Upon-Tyne, NE1 7RU, United Kingdom

^c Scientific Computing Department, Science and Technology Facilities Council, Daresbury Laboratory, Warrington, WA4 4AD, United Kingdom

ARTICLE INFO

Article history:

Received 6 July 2021

Received in revised form 7 April 2022

Accepted 12 July 2022

Available online 16 July 2022

Keywords:

Adaptive mesh refinement

Morton code

Polynomial flux reconstruction

Runge-Kutta scheme

Reacting flows

Direct numerical simulations

ABSTRACT

A new Direct Numerical Simulation (DNS) code HAMISH with Adaptive Mesh Refinement (AMR) has been developed to simulate compressible reacting flow in a computationally economical manner. The focus is on problems where high gradients of temperature, density and species mass fraction remain localised, for example within the interior structure of flames. The resolution requirements of such local high-gradient regions often determine the global mesh spacing when uniform meshes are used. The numerical framework in HAMISH is based on an unstructured finite-volume approach together with a Runge-Kutta algorithm for time-stepping. The unstructured Cartesian mesh used in HAMISH allows for adaptive refinement and de-refinement of the local mesh depending on the demands of the flow physics and numerical accuracy. The code utilises an octree data structure and a Morton space-filling curve, allowing efficient cell division as well as straightforward parallel domain decomposition. The spatial discretisation scheme uses fourth-order polynomial reconstruction to evaluate the inter-cell fluxes. A third-order three-step explicit Runge-Kutta time-marching algorithm is used together with adaptive time-stepping with proportional-integral-type error control using an embedded Runge-Kutta scheme. The code is fully parallelised through domain decomposition over an arbitrary number of processors and exhibits a high level of parallel efficiency. In this paper the key capabilities of HAMISH are demonstrated based on a number of test cases. These test cases include (a) simulation of 1-D planar laminar premixed flames to demonstrate the capability of AMR in capturing the sharp gradients within the flame; (b) fully developed laminar channel flow, which shows that the code is capable of refining the mesh in the boundary layer next to the wall; (c) 2-D expanding flame under quiescent laminar condition, which demonstrates that the code is capable of dynamic local mesh refinement based on the position of the flame front; (d) 3-D non-reacting Taylor-Green vortex which demonstrates HAMISH can deal with vortical motion typical of turbulent flows; (e) 3-D premixed turbulent flame propagation under isotropic homogeneous decaying turbulence to demonstrate that the code can be employed for production level DNS of turbulent reacting flows.

© 2022 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

* Corresponding author.

E-mail address: rs10@cam.ac.uk (R.S. Cant).

<https://doi.org/10.1016/j.jcp.2022.111480>

0021-9991/© 2022 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

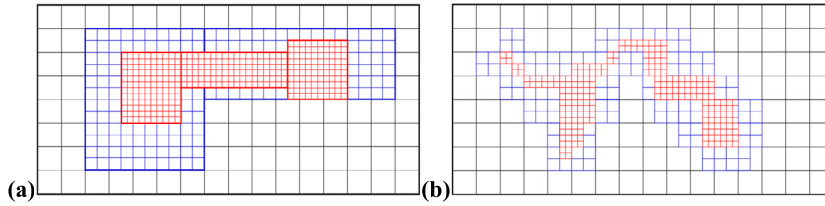


Fig. 1. Examples of data structures used by AMR codes. (a): block-structured, (b) point-structured.

1. Introduction

In combustion the chemical reaction often takes place in a thin sheet-like layer separating reactants and products [1]. Hence, for accurate simulation of combustion it is necessary to resolve the thin reaction layer and the associated strong gradients of temperature and chemical species mass fraction. Typically, about 10–20 mesh points are required within the reaction layer in order to resolve key thermo-chemical processes within the flame, and the mesh resolution requirements become increasingly stringent with increasingly complex chemical reaction mechanisms [2]. These requirements can be even more challenging in turbulent reacting flows because the smallest length scales of turbulence (i.e. the Kolmogorov length scale) also needs to be resolved. Even greater spatial resolution is required in cases such as local ignition, flame-wall interaction (FWI) and high-pressure combustion since the flame thickness is known to decrease with increasing pressure [3].

In order to carry out detailed simulations of the flow and thermochemistry of combustion within reasonable computational costs, a dynamic adaptive mesh refinement (AMR) approach is desirable. By definition, an AMR approach refines and de-refines the mesh based on the local properties of the solution as it evolves in time, so that a fine mesh can be used only where required, e.g. within the reaction layer of the flame. Since the flame is evolving in time due to convection and propagation, its location within the computational domain is not known a priori. Hence, using a non-adaptive mesh is much less computationally cost-effective since the finest mesh must be applied everywhere in the computational domain in order to capture the motion of the flame. Similar considerations apply in the case of FWI, where the high gradients experienced within the flame close to the wall must be resolved using a very fine mesh. Moreover, the size of the smallest grid spacing is often not known a priori. Using AMR allows the mesh to follow the flame and removes the need for a fine mesh to be maintained statically close to the wall. Similarly, during a local ignition event, the maximum temperature of the burned gas can attain large super-adiabatic values [4], which leads to thin and highly transient layers with steep temperature gradients. These must be resolved using a fine mesh, and again the use of AMR avoids the need for a fine mesh everywhere, and at all times.

The advantages of AMR can make it possible to conduct high-fidelity simulations where the use of a fixed mesh would be prohibitively expensive. An example is Direct Numerical Simulation (DNS) of turbulent combustion, where it is often the case that a much finer mesh is required to resolve the flame structure than is required to resolve the smallest scales of turbulence. Hence it is possible using AMR to achieve higher turbulent Reynolds numbers and carry out simulations of more complex configurations. Thus, the development of an AMR approach has the potential to offer a step-change in the computational analysis of reacting flows.

The AMR approach has been extensively used in astrophysics, detonation modelling, and to capture interfaces in two-phase flow [5–7]. In these application areas, AMR has been shown to enhance the spatial resolution dynamically at low computational cost. Another major advantage of AMR is that the computational mesh is generated automatically where needed, and similar techniques have been used in mesh generation software [8]. Many current AMR codes used rely on block-structured data (Fig. 1a) or pointwise-structured data (Fig. 1b). Examples of structured data libraries/codes are BoxLib [9], Chombo [10], and Paramesh [11]. The advantage of a structured-data solver is that the calculation of numerical fluxes or differences can be conducted on cells of the same level and therefore special treatment for non-conformal mesh or hanging nodes can be avoided. However structured-data AMR methods increase numerical complexity due to the overlapping patches required by the different resolutions, especially for large-scale parallel computation in which the overhead for parallel communication within and between layers hinders the performance. An example using block-structured data is the reacting-flow code LMC (Low-Mach Combustion), [12], while an example using point-structured data is the two-phase flow code Gerris [13]. Both of these codes are restricted to low Mach number flows. Unstructured-data AMR algorithms based on space-filling Morton/Peano [14,15] or Hilbert curves [16], and making use of the octree data structure [17] provide a powerful option to develop efficient unstructured-data AMR codes. These algorithms have been shown to achieve good parallel performance and scalability [18,19]. The unstructured-data AMR strategy allows for greater flexibility to handle complex geometries and high-order accuracy by storing all the neighbourhood links between cells [20–22]. The present paper describes the AMR based compressible reacting-flow DNS code HAMISH, which makes use of the Morton space-filling curve and octree data structure.

The numerical framework of HAMISH relies on a finite-volume approach for spatial discretisation together with a Runge-Kutta algorithm for time-stepping. An unstructured Cartesian mesh allows for adaptive mesh refinement and de-refinement operations based on the local resolution requirements prescribed by the numerical solution. The data structure is built by

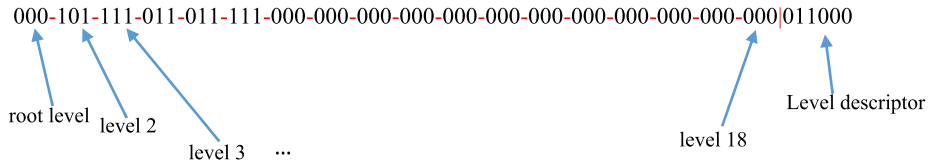


Fig. 2. Structure of a Morton code.

representing the unstructured mesh as an octree whose leaves correspond to the individual cells of the mesh. The cells are indexed with the help of a Morton space-filling curve, allowing efficient octree searching as well as straightforward parallel domain decomposition. The spatial discretisation scheme evaluates the inter-cell fluxes by reconstructing the solution in each cell using a polynomial approximation. The order of the scheme is set by the degree of the polynomial, and an arbitrarily-high order can be attained in principle. In practice, fourth-order reconstruction [23,24] offers a reasonable compromise between accuracy and computational cost. For time-stepping, the Runge-Kutta algorithm [25] may be adjusted to have any number of sub-steps in order to achieve the required order of accuracy, with adaptive time-stepping using error control from an embedded Runge-Kutta scheme. The standard time-stepping algorithm is a third-order three-step Runge-Kutta scheme with a second-order embedded scheme together with a proportional-integral (PI)-type time step controller. The code is fully parallelised through domain decomposition with dynamic load-balancing over an arbitrary number of processors.

The main objectives of this paper are to present the underlying numerical methods of the HAMISH code and to demonstrate the key capabilities of the code based on the following examples:

- (a) Simulation of 1-D planar laminar premixed flames to demonstrate the capability of AMR in capturing the sharp gradients within the flame.
- (b) Fully developed laminar channel flow, which shows that HAMISH is capable of refining the mesh in the boundary layer next to the wall.
- (c) 2-D expanding ignition front under quiescent laminar condition, which demonstrates the capability of HAMISH to refine the mesh dynamically based on the position of the ignition front.
- (d) 3-D non-reacting Taylor-Green vortex, which demonstrates HAMISH can deal with vortical motion typical of turbulent flows.
- (e) 3-D premixed turbulent flame propagation under isotropic homogeneous decaying turbulence to demonstrate that the code can be used for DNS of turbulent reacting flows.

The numerical methods pertaining to HAMISH are presented and discussed in the next section of this paper. Following that, the results for the representative cases are shown and the associated code performance is discussed. Finally, the major findings are summarised, and conclusions are drawn.

2. Mathematical and numerical framework

Morton codes and Adaptive Mesh Refinement (AMR)

The Navier-Stokes equations for multi-component variable-property gas mixtures are discretised using a conservative finite-volume method on an unstructured mesh consisting of cubic cells. The conserved solution variables are ρ , ρu_i , ρE and ρY_α , where ρ is the density, u_i is the velocity component in the i^{th} direction, E is the specific total internal energy and Y_α is the mass fraction of species α . Cell-averaged values of each of the conserved variables are stored at the cell centres. Every cell has an index number, and is also assigned a unique Morton code, which serves three different purposes. Firstly, the Morton codes map directly onto an octree data structure, which enables rapid searching for any cell in the mesh. Secondly, the Morton code provides a concise set of instructions for traversing the octree from its root to the corresponding cell. Thirdly, the Morton code encapsulates the three integer spatial coordinates of the cell, allowing immediate conversion from physical location to Morton ordering and vice versa. Each Morton code is globally unique, and the list of Morton codes – which in general is not contiguous – is stored in ascending order and forms a space-filling curve that maps the three-dimensional physical space onto a one-dimensional computational space. The list of Morton codes is distributed in ascending order across the available processors, providing a natural parallelisation strategy. The corresponding octree data structure is distributed in the same manner. On each processor the local list of Morton codes is mapped one-to-one onto the cell index numbers, which are purely local to that processor.

In the present case the Morton code corresponding to a cell is defined as a bit-string of eighteen 3-bit octants together with a 6-bit level descriptor, and an example is shown in Fig. 2.

The octal numbers in the Morton code from the left-hand end up to the highest level indicated by the level descriptor can be interpreted as a set of instructions giving the path through the octree from the root to the specified cell. This allows for rapid searching of the octree, for example to find the index number of a cell whose integer coordinates are known. This

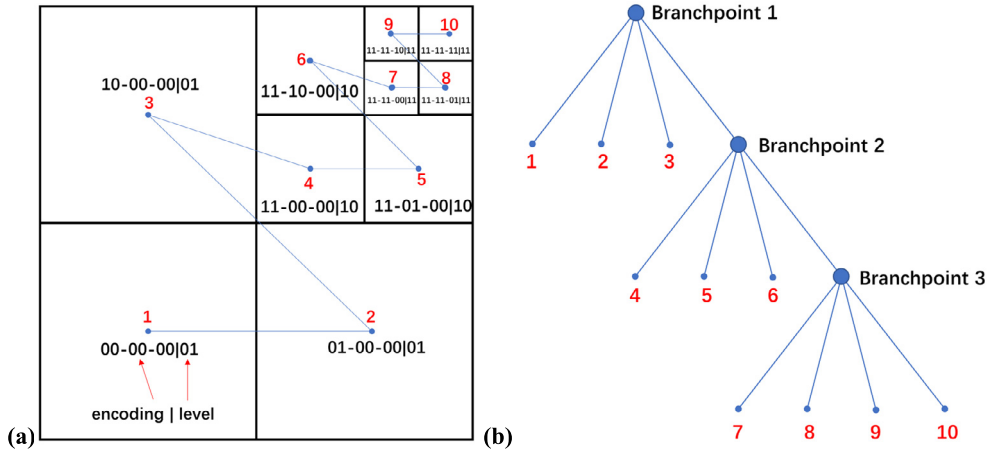


Fig. 3. (a) Cell ordering and Morton codes for a 2-D AMR mesh and (b) its corresponding quadtree structure.

also ensures that the octree and the list of Morton codes are effectively interchangeable, and one may be generated from the other. A simplified example is shown in Fig. 3 for a 2-D case, as described by a quadtree, and the extension to the 3-D case with an octree is straightforward.

In three dimensions, the cells are visited in a Z-ordered sequence in which each cell in a group of eight cells is visited starting with the cell having the lowest x - y - z coordinates, moving first in the x -direction, then in the y -direction and then in the z -direction, returning to the lowest value of the previous coordinates each time the direction is altered.

A convenient property of the Morton code is that the three spatial coordinates of the cell can be obtained by extracting the bit string corresponding to the first bit of each octant to give the z -coordinate, the second bit to give the y -coordinate and the third bit to give the x -coordinate. The coordinates expressed in this manner are the integer coordinates of the lowest-order (Down-South-West = DSW) corner of the cell, starting from the origin in the DSW corner of the computational domain, and measured in integer units of the size of the cell. Each Morton code as defined here may be stored conveniently within a single 8-byte integer number and manipulated using high-level bit operations.

The root of the octree structure at level 0 corresponds to the Big Cube, which is defined as the smallest cube that encompasses the entire computational domain. Then level 1 is taken to be the eight equally-sized cubes inside the Big Cube. At every subsequent level, the cubes at the previous level are divided into eight equally sized smaller cubes. An initial mesh may be generated by specifying the level matching the required initial cell size. The cells of this mesh are distributed automatically and as evenly as possible across the available processors. The corresponding octree is distributed such that every processor has its own copy of the root, together with just enough of the global octree to connect the root to the local portion of the mesh. Branches which lead to parts of the octree that are located on different processors are flagged as remote and are handled using parallel data transfers.

The AMR procedure is based on a set of refinement and de-refinement criteria dictated by the physics of the flow solution. An AMR step begins with evaluation of the criteria and the generation of two lists of cells marked respectively for refinement and de-refinement. Mesh refinement is carried out by splitting a cubic parent cell of side h into 8 cubic child cells of side $h/2$. By contrast, de-refinement is carried out when 8 child cells of side $h/2$ are combined to form a parent cell of side h . De-refinement takes precedence over refinement in order to reduce peaks in memory usage.

Cells marked for de-refinement are assessed to ensure that they form groups of eight that are all children of a single parent cell. Cells not meeting this requirement are removed from the de-refinement list. De-refinement is effected by first reducing the level of the lowest Morton code in the octant by one. The corresponding cell is marked as a de-refined parent cell while the others in the octant are marked as de-refined child cells. When all cells in the list have been marked, garbage collection is carried out by removing the redundant child-cell Morton codes and closing up the Morton code list.

Refinement of a cell begins when its Morton code is modified by incrementing the level descriptor by 1 and ensuring that the new octant at that level is set equal to zero. Then seven new Morton codes are created each having the new level descriptor, each inheriting the octant sequence of the parent together with a new octant having the correct sequence number. Starting with the Morton code shown in Fig. 2, the set of new Morton codes following refinement is displayed in Fig. 4.

After refinement of all relevant Morton codes, the octree is regenerated, and new cells are created. At this point, solution values of the conserved variables are allocated to the cells using a conservative interpolation scheme compatible with the spatial discretisation scheme (see later).

Refinement and de-refinement lead to transitions in cell size between adjacent cells. To minimise the number of special cases required by the spatial discretisation scheme, only one level of refinement is permitted between adjacent cells. Hence, $h - 2h$ size transitions are allowed, but transitions of $h - 4h$ or greater are not. This is ensured by balancing the octree after each refinement or de-refinement operation. Tree balancing is achieved by searching (using the current octree) for the



Fig. 4. Refinement of a Morton code in 3-D.

six geometrical face-neighbours of each cell in the mesh. Any neighbour cell that is found to be more than one level less refined is marked for refinement. Once the neighbour search is completed for all cells in the mesh, the refinement operation described above is carried out on all marked cells, and the process is repeated in an iterative manner until there are no cells left to refine. This ensures that there are no transitions greater than $h - 2h$ and the tree is balanced.

As a result of refinement and de-refinement, the number of cells allocated to each processor may change, and this is likely to give rise to an imbalance of computational work between processors. This is addressed using a dynamic load-balancing procedure, which allows for reallocation of cells between adjacent processors until an equitable distribution is obtained. Since the cells and processors are arranged in ascending Morton code order, the load-balancing problem is one-dimensional and is solved using a diffusion process. For a given number of cells N_p on each processor p , a discrete diffusion equation may be written as:

$$N_p^{m+1} = N_p^m + \alpha_N \left[\left(N_{p+1}^m - N_p^m \right) - \left(N_p^m - N_{p-1}^m \right) \right] \quad (1)$$

where the parameter α_N is an effective diffusivity and m is an iteration index. The stability limit for the iteration is given by $\alpha_N = 0.5$. Equation (1) indicates that the diffusion of cells is determined by the difference in the number of cells between adjacent processors, and the stability limit indicates that the maximum number of cells transferred in any one iteration is equal to half of the difference. The iterative process is continued until the load is balanced to the required level of accuracy. It is worth noting that this process requires data transfers between nearest-neighbour processors only.

Spatial discretisation

The spatial discretisation scheme requires a stencil to be generated for each cell in the mesh and the size of the stencil depends on the order of accuracy of the spatial discretisation scheme. Cell index numbers of all the cells in the stencil are stored in a stencil list for each cell in the mesh. The stencil list is built recursively by accumulating successive layers of cells that are face-neighbours of the cells in the previous layer. For the first layer, the previous layer contains only the central cell itself. For each cell in the previous layer, all six faces are visited in turn. The neighbour Morton code is constructed for each neighbouring cell and a search of the octree is carried out. There are five valid outcomes from the search:

- **Clean face:** The neighbouring cell resides inside the computational domain, is located on the same processor, and has the same size h as the previous-layer cell. The neighbouring cell index is added to the stencil list.
- **$h - 2h$ transition:** The neighbouring cell resides inside the computational domain, is located on the same processor, and has the twice the size $2h$ as the previous-layer cell. The neighbouring cell index is added to the stencil list.
- **$2h - h$ transition:** The neighbouring cell resides inside the computational domain, is located on the same processor, and has half the size as the previous-layer cell. Of the eight sub-cells in the subdivided neighbour, four are direct face-neighbours of the previous-layer cell. All four Morton codes are constructed in turn, and the cell index for each is added to the stencil list.
- **Remote cell:** The neighbouring cell is allocated to a different processor. The neighbour Morton code is stored in a remote-cell list for further investigation.
- **Boundary face:** A new boundary condition cell index is generated.

After the search has been completed for every cell in the previous layer, a parallel transfer takes place and any remote cells are reconciled. Each stencil list is then sorted in Morton code order and any duplicate cells are removed.

During the stencil generation procedure, a face-neighbour list is created for every central cell. This contains six entries indexed by the direction of the outward-pointing normal vector for each face: West (W), East (E), South (S), North (N), Down (D), Up (U). Each entry in the face-neighbour list is a cell index number, and the face-neighbour list is similar to the first layer of the stencil. In the case of a $2h-h$ transition, the cell index number of the face-neighbour cell with the lowest

Table 1
Monomials ψ_k for a fourth order reconstruction ($p = 3$).

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
x	x^2	x^3	y	yx	y^2	yx^2	y^2x	y^3	z	zx	zy	z^2	zx^2	zxy	z^2x	zy^2	z^2y	z^3

Morton code is stored. The other three cell index numbers are found using their own face-neighbour lists with the help of a connectivity table.

Flux evaluation

The inter-cell fluxes are evaluated using polynomial reconstruction of the conserved variables, which are stored at cell centres. Fourth order reconstruction has been found to offer a good compromise between accuracy and computational cost. In this approach, a variable $u(x, y, z)$ can be expressed as [23,24]:

$$u(x, y, z) = \bar{u}_0 + \sum_{k=1}^K a_k^{(u)} \phi_k(x, y, z) \quad (3)$$

where u is the solution variable, $\bar{u}_0$ is the cell-averaged value of that variable in the central cell, $a_k^{(u)}$ is the k^{th} coefficient of the polynomial for the solution variable u , and the basis function ϕ_k is given by:

$$\phi_k = \psi_k - \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} \psi_k dx dy dz \quad (4)$$

where ψ_k is the k^{th} monomial. Here, x , y and z are scaled coordinates based on the size h_0 of the central cell. The number of monomials K is given by:

$$K = p(1-D); \quad K = \frac{1}{2}(p+1)(p+2) - 1 \quad (2-D); \quad K = \frac{1}{6}(p+1)(p+2)(p+3) - 1 \quad (3-D) \quad (5)$$

For fourth order accuracy a polynomial of degree $p = 3$ is required and the number of monomials $K = 19$ in three dimensions. The monomials are listed in Table 1.

The basis functions ϕ_k are the same as the monomials ψ_k except for three special cases:

$$\phi_2 = x^2 - 1/12; \quad \phi_6 = y^2 - 1/12 \quad \text{and} \quad \phi_{13} = z^2 - 1/12 \quad (6)$$

The coefficients $a_k^{(u)}$ are obtained using the cell-averaged values $\bar{u}_j$ in the central cell ($j = 0$) and J other cells forming the stencil. It is a necessary that $J \geq K$, and in addition the stencil must have enough cells in each direction to support the required order of accuracy in all of them. In practice, it is desirable to take $K \leq J \leq 2K$ and for the stencil to be as isotropic as possible [26]. Integration of the polynomial over cell j of the stencil yields:

$$\bar{u}_j = \bar{u}_0 + \sum_{k=1}^K a_k^{(u)} A_{jk} \quad (7)$$

The elements of the non-square matrix A_{jk} are given by integrals of the basis functions as:

$$A_{jk} = \frac{1}{\hat{h}_j^3} \int_{-\hat{h}_j/2}^{\hat{h}_j/2} \int_{-\hat{h}_j/2}^{\hat{h}_j/2} \int_{-\hat{h}_j/2}^{\hat{h}_j/2} \phi_k(x - \hat{x}_j, y - \hat{y}_j, z - \hat{z}_j) dx dy dz \quad (8)$$

where $\hat{h} = h_j/h_0$ is the relative size of the j^{th} cell and $(\hat{x}_j, \hat{y}_j, \hat{z}_j)$ is the location of its centre, expressed in the scaled coordinate system of the central cell. Note that A_{jk} contains purely geometrical information about the local mesh and requires no information from the solution. The coefficients a_k^u are found by solving the linear system $A_{jk} a_k^u = b_j^u$ where $b_j^u = \bar{u}_j - \bar{u}_0$. The solution is found using Singular Value Decomposition (SVD) to obtain the Moore–Penrose pseudo-inverse matrix A_{jk}^+ . Since A_{jk} contains only geometrical information, the pseudo-inverse needs to be computed only once for each stencil. Finally, the coefficients for each solution variable are obtained using $a_k^u = A_{jk}^+ b_j^u$.

Flux treatment

The governing conservation equations are integrated over each cell and take the form:

$$\partial \bar{U} / \partial t = \bar{S}(\bar{U}, t) + \frac{1}{h} \sum_{k=1}^6 \bar{F}_k(\bar{U}) \quad (9)$$

where $\bar{U} = \{\bar{\rho}, \bar{\rho}u_i, \bar{\rho}E, \bar{\rho}Y_\alpha\}$ is the vector of cell-averaged conserved variables, $\bar{S} = \{0, 0, 0, 0, 0, \bar{w}_\alpha\}$ is the vector of cell-averaged source terms and $\bar{F}_k$ is the vector of the face-averaged fluxes on face k of the cell. The cell-averaged reaction rates are given by:

$$\bar{w}_\alpha = \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} w_\alpha dx dy dz \quad (10)$$

where the integral is evaluated numerically using Gaussian quadrature.

The fluxes $\bar{F}_k$ can be split into face averaged convective and diffusive contributions as $\bar{F}_k = \bar{D}_k - \bar{C}_k$ with

$$\bar{C}_k = \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} C_k dA_k; \quad \bar{D}_k = \int_{-1/2}^{1/2} \int_{-1/2}^{1/2} D_k dA_k \quad (11)$$

Here, dA_k is the element of area on the cell face k and the convective and diffusive contributions C_k and D_k are given by:

$$C_k = \begin{bmatrix} \rho u_k \\ \rho u_k u_i - p \delta_{ki} \\ \rho u_k E + p u_k \\ \rho u_k Y_\alpha \end{bmatrix}; \quad D_k = \begin{bmatrix} 0 \\ \frac{\partial \tau_{ki}}{\partial x_k} \\ -\frac{\partial q_k}{\partial x_k} + \frac{\partial \tau_{km} u_m}{\partial x_k} \\ -\frac{\partial (\rho V_{\alpha,k} Y_\alpha)}{\partial x_k} \end{bmatrix} \quad (12)$$

where u_k, q_k and $V_{\alpha,k}$ are the k^{th} components of velocity, heat flux and diffusion velocity of species α , respectively, p is the thermodynamic pressure, and τ_{ki} is the component of viscous stress tensor. The integrals in eq. (11) are evaluated numerically by Gaussian quadrature.

Cases of $h - 2h$ cell size transitions are handled by subdividing the face of the larger cell into four sub-faces and evaluating the flux separately for each sub-face. The Gaussian integration procedure requires values of the relevant physical quantities at each Gauss point. For the conserved quantities these point values are found by direct evaluation of the reconstructed polynomials (eq. (3)). Primitive variables are evaluated locally from the conserved quantities at each Gauss point in order to preserve the order of accuracy of the spatial reconstruction scheme. Point values of the derivatives, as required by the diffusive fluxes, are obtained by differentiating the reconstructed polynomials for the conserved quantities, and extracting the derivatives of primitive quantities at each point using algebraic expressions:

$$\begin{aligned} \frac{\partial}{\partial x_j} u_i &= \frac{1}{\rho} \left[\frac{\partial (\rho u_i)}{\partial x_j} - u_i \frac{\partial \rho}{\partial x_j} \right]; & \frac{\partial}{\partial x_j} Y_\alpha &= \frac{1}{\rho} \left[\frac{\partial (\rho Y_\alpha)}{\partial x_j} - Y_\alpha \frac{\partial \rho}{\partial x_j} \right]; \\ \frac{\partial}{\partial x_j} T &= \frac{1}{\rho C_v} \left[\frac{\partial (\rho E)}{\partial x_j} - \sum_{\alpha=1}^N \left(h_\alpha - \frac{R^0 T}{W_\alpha} \right) \frac{\partial (\rho Y_\alpha)}{\partial x_j} - u_k \frac{\partial (\rho u_k)}{\partial x_j} + \frac{1}{2} u_k u_k \frac{\partial \rho}{\partial x_j} \right] \end{aligned} \quad (13)$$

where R^0 is the universal gas constant and W_α is the molecular weight of species α . The fluxes for each cell face are computed from the reconstructed values obtained from both the left and the right sides of each cell face. These left (L) and right (R) values for each Gauss point are not necessarily equal to each other. For the diffusive fluxes, the L and R values of the flux are computed separately at each Gauss point and averaged. For the convective fluxes, the implied discontinuity between the L and R thermochemical states across the cell face is treated using a Riemann solver at each Gauss point [27]. A linearised Riemann solver is found to be sufficient for the low Mach numbers of interest in most turbulent combustion problems.

It is worth noting that the spatial discretisation scheme was selected partly for its computational economy in terms of both memory and CPU time, but mainly for its ability to deal with very rapid changes of mesh size, i.e. $h - 2h$ transitions between adjacent cells, and its ability to deal with hanging nodes at the $h - 2h$ cell interfaces. High order (greater than 2nd order) was regarded as essential for DNS, not only in the highly-resolved chemically-reactive regions but also in the less well-resolved far field where turbulence still needs to be properly represented. A similar approach called CENO (Central Essentially Non-Oscillatory) [24] has been used in somewhat different applications and on somewhat different meshes.

Finally, boundary conditions are specified by setting values of the primitive variables at the Gauss points on the faces of boundary cells. These values may be specified directly, for example on inflow boundaries where the velocity components are set together with either the density or the temperature. Alternatively, boundary values may be derived using the Navier-Stokes Characteristic Boundary Condition (NSCBC) technique [28,29], for example on non-reflecting outflow boundaries.

Time-advancement

The solution is advanced in time using an explicit Total Variation Bounded (TVB) Runge-Kutta method [25] designed for use with WENO-type reconstruction schemes. A PI-based adaptive time-step controller has been added [30]. The new time step $\Delta t^{(n+1)}$ is given by:

$$\Delta t^{(n+1)} = \kappa \Delta t^{(n)} \left(\frac{\epsilon}{\|E^{(n+1)}\|_\infty} \right)^\alpha \left(\frac{\|ER^{(n)}\|_\infty}{\epsilon} \right)^\beta \quad (14)$$

where ER is an error estimate obtained dynamically using a second-order embedded Runge-Kutta scheme, ϵ is a pre-set error tolerance, α and β are the parameters of the controller and κ is a safety factor. The controller requires some tuning for optimal performance, but acceptable results have been obtained with $\epsilon = 1 \times 10^{-3}$, $\kappa = 0.5$, $\alpha = 0.6/p$ and $\beta = 0.2/p$ where $p = 2$ is the order of the embedded scheme.

3. Results and discussion

1-D Laminar premixed flame solution

Numerical simulations of 1-D freely-propagating laminar stoichiometric methane-air and hydrogen-air premixed flames have been carried out using HAMISH with and without AMR. When using AMR, the refinement criteria are based on the gradients of velocity, temperature and species mass fraction. For methane-air flames, a skeletal chemical mechanism involving 16 species and 25 reactions [31] has been used together with mixture-averaged formulation for molecular transport. For the purpose of comparison, simulations of the same flames were conducted using the well-established finite difference combustion DNS code SENG+ [32,33] with uniform mesh spacing and the same chemical mechanism. In SENG+, the spatial derivatives are evaluated using a 10th order central difference technique gradually reducing to a one-sided 4th order scheme at non-periodic boundaries. Time-advancement in SENG+ is carried out using a low storage 4th order Runge-Kutta scheme. Non-periodic boundaries in SENG+ are specified using the NSCBC methodology [28,29].

The variations in the direction of flame propagation (taken as the x -direction) of gas density ρ , temperature T , velocity in the flow direction u , mass fractions of the major species methane, oxygen, carbon dioxide and water vapour (Y_{CH_4} , Y_{O_2} , Y_{CO_2} , Y_{H_2O}), together with the representative intermediate species carbon monoxide (Y_{CO}) are shown in Fig. 5 for a freely propagating 1-D stoichiometric methane-air premixed laminar flame. The results for HAMISH without AMR (blue dashed line) and SENG+ (red dashed line) are obtained using a mesh of 2,048 uniformly-spaced points in a domain size of 10^{-2} m. The results for HAMISH with AMR (black line) using refinement criteria based on scalar, temperature and velocity gradients reduce the mesh requirements to 147 non-equidistant mesh points.

Fig. 5 indicates that the distributions of density, temperature, and species mass fractions from HAMISH (with and without AMR) are in excellent agreement with the corresponding results obtained from SENG+. Indeed, the results from these two codes cannot be distinguished from each other. The results from both HAMISH and SENG+ are further compared to results from the open-source flame code Cantera [34] (see: <https://www.cantera.org>) employing a detailed chemical mechanism (GRI-Mech 3.0) [35] (purple dotted line) along with mixture-averaged transport and also using a skeletal chemical mechanism involving 16 species and 25 reactions [31] (green dotted line), which yielded a 1-D laminar premixed flame solution on 545 mesh points using AMR for GRI-Mech 3.0 and 502 mesh points for the skeletal chemical mechanism. It can be seen from Fig. 5 that the results obtained from HAMISH and SENG+ with the skeletal mechanism are in good qualitative and quantitative agreement with the detailed chemistry results obtained from Cantera. The remaining quantitative differences between the distributions of intermediate species can be attributed to the differences between the chemical reaction mechanisms. The unstrained laminar burning velocity S_L obtained from HAMISH with and without AMR has the same value of 0.415 m/s. The same quantity obtained from SENG+ and Cantera for the skeletal mechanism [31] is found to be larger by 1.20% and 3.61% respectively. When using HAMISH with AMR, the saving is CPU time relative to HAMISH without AMR is found to be 81.5%.

The HAMISH AMR mesh with non-uniform spacing for the 1-D flame case of Fig. 5 is shown in Fig. 6 superimposed on the distribution of temperature T . It is evident that there is a high degree of mesh refinement corresponding to the strong temperature gradient around the flame location.

The satisfactory results of HAMISH are not only limited to methane-air flames and similar results are also obtained for 1-D simulations of laminar stoichiometric hydrogen-air premixed flames. Here, the computations used a skeletal mechanism involving 9 species and 21 reactions [36], together with mixture-averaged transport. Fig. 7 shows the variations with distance through the flame of density ρ , temperature T , velocity in the flow direction u_1 and the mass fractions of molecular

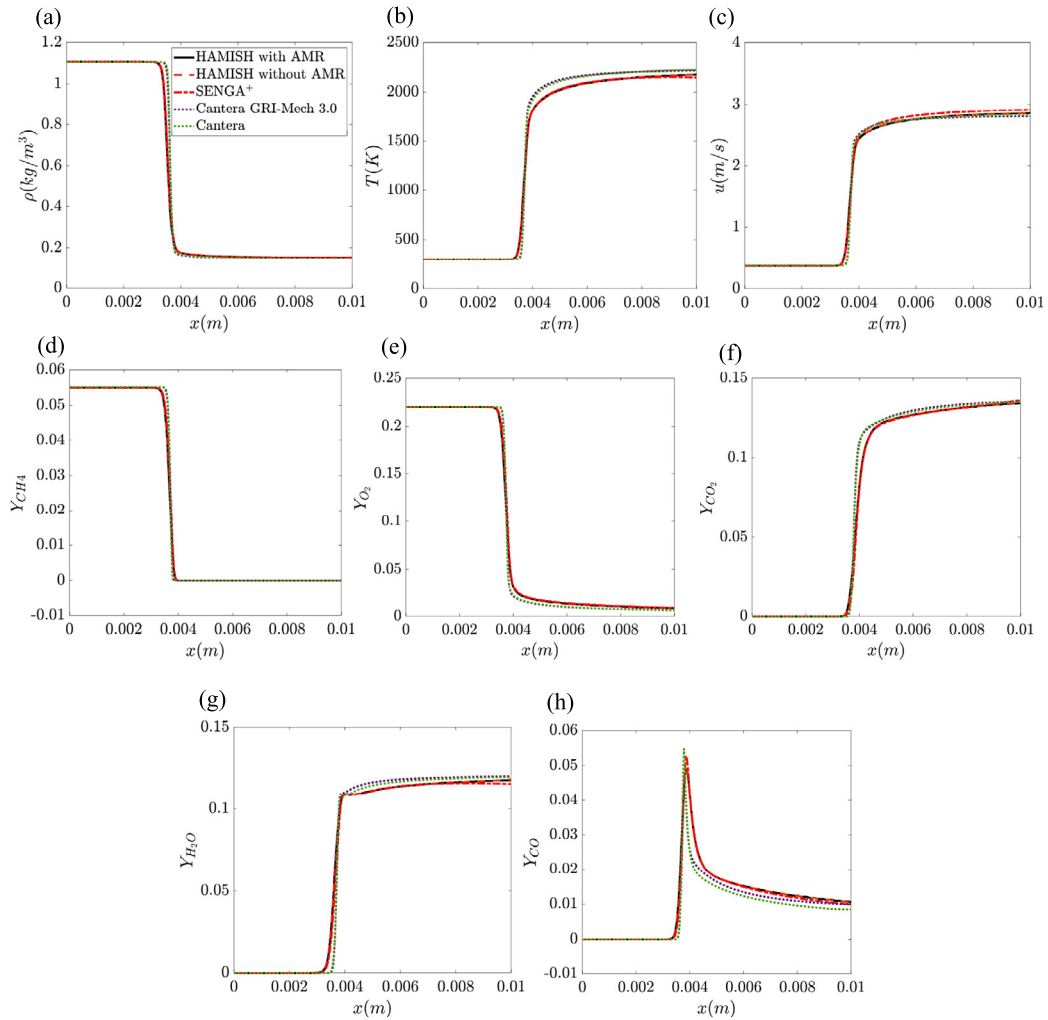


Fig. 5. Variations with distance x through the flame of (a) gas density ρ , (b) temperature T , (c) velocity in the flame-normal direction u , (d-h) mass fractions of methane, oxygen, carbon dioxide, water vapour and carbon monoxide for a 1-D freely propagating stoichiometric CH_4 -air laminar premixed flame as obtained from SENG+ and from HAMISH with and without AMR using a skeletal chemical mechanism [31]. Results from Cantera using the full GRI-Mech 3.0 mechanism are also included. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)

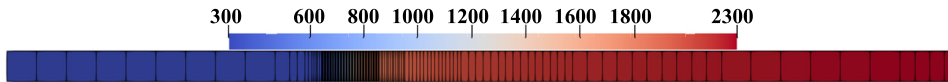


Fig. 6. The HAMISH AMR mesh superimposed on the distribution of temperature T in Kelvin (K) for the 1-D freely propagating stoichiometric CH_4 -air laminar premixed flame simulation reported in Fig. 5.

hydrogen, oxygen, water vapour (as major species), and atomic hydrogen and hydroxyl radical (as representative intermediate species). Once again, there is excellent agreement between the results from HAMISH without and with AMR, and with SENG+ using the same treatment for chemistry and molecular transport. The results in Fig. 7 for HAMISH without AMR (blue dashed line) and for SENG+ (red dashed line) are obtained using 2,048 mesh points for a domain size of 10^{-2} m. The use of HAMISH with AMR (black line) reduces the mesh requirements to 243 non-equidistant mesh points. The results obtained from HAMISH and SENG+ are also found to be in good agreement with results for the same flame case from Cantera (dotted line) based on the GRI-Mech 3.0 full mechanism [35], which are obtained on 509 mesh points using AMR. The minor differences in the distributions of intermediate species between HAMISH/SENG+ and Cantera are again attributed to the differences in chemical reaction mechanisms.

For the hydrogen flame, the unstrained laminar burning velocity S_L obtained from HAMISH with and without AMR has the same value of 2.17 m/s. The laminar burning velocity obtained from SENG+ and Cantera is found to be larger by 5.53% and 7.37% respectively. When using HAMISH with AMR, the saving in CPU time relative to HAMISH without AMR is found to be 75.7%.

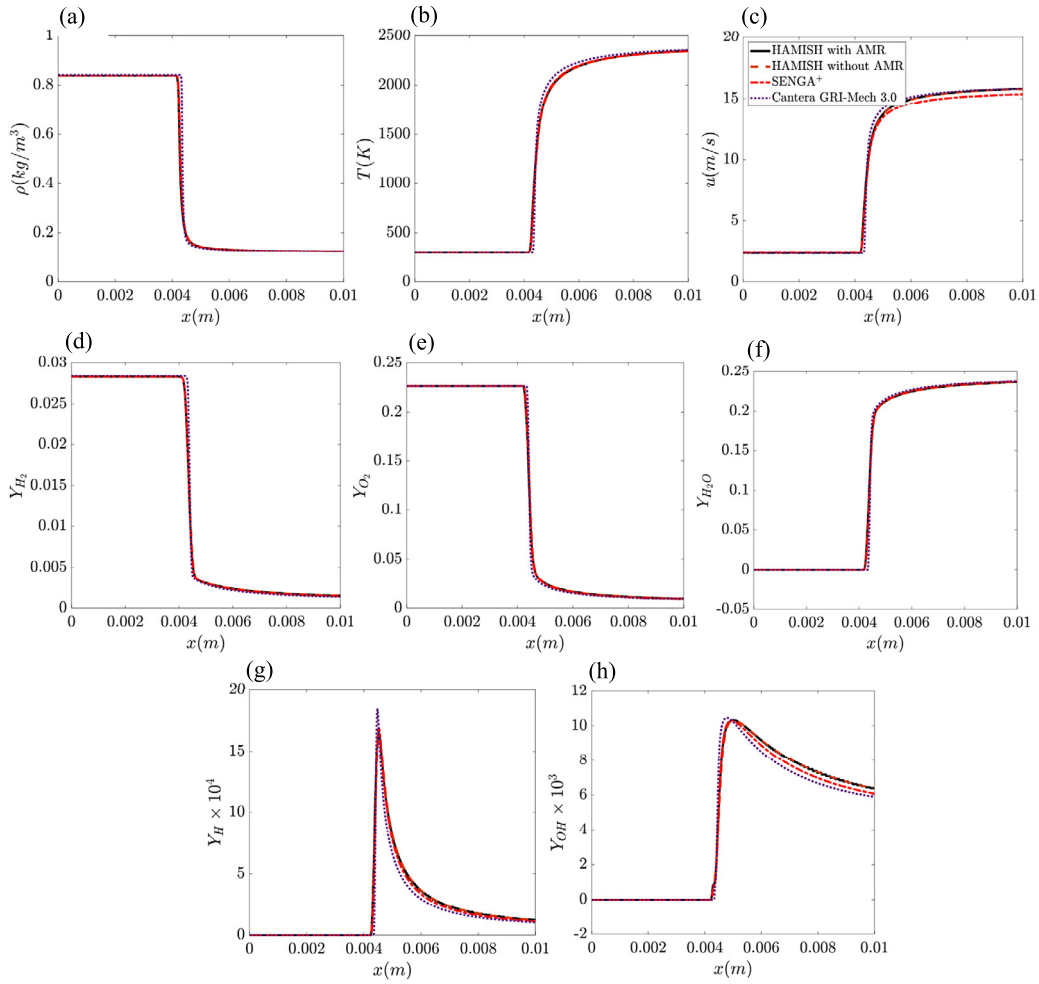


Fig. 7. Variations with distance x through the flame of (a) gas density ρ , (b) temperature T , (c) velocity in the flame-normal direction u , (d-h) mass fractions of molecular hydrogen, oxygen, water vapour, atomic hydrogen and hydroxyl radical for a 1-D freely propagating stoichiometric H_2 -air laminar premixed flame as obtained from SENG+ and from HAMISH with and without AMR using a skeletal chemical mechanism [36]. Results from Cantera using the full mechanism GRI-Mech 3.0 are also included.

Laminar fully developed channel flow

In order to demonstrate that HAMISH with AMR based on the velocity gradient is capable of refining the mesh in the boundary layer next to the wall in a fully-developed laminar channel flow, a simulation has been conducted using HAMISH for a friction Reynolds number of $Re_\tau = u_\tau h / \mu = 64$ where h is the channel half height, μ is the viscosity and $u_\tau = \sqrt{|\tau_w|/\rho}$ is the friction velocity with τ_w being the wall shear stress. For the purpose of comparison, the same simulation was conducted with SENG+ and also using the well-known finite-volume solver Code_Saturne [37] (see: <http://www.code-saturne.org/>). The simulations were initiated with 8,000 (20×400) equidistant mesh points. Using HAMISH with AMR gave rise to a mesh of 3200 cells with near wall mesh spacing ensuring $y^+ = \rho u_\tau \Delta y / \mu = 0.6$ (where Δy is the near-wall mesh spacing) adjacent to the wall. The boundaries in the x -direction are taken to be periodic and the boundaries in the y -direction are taken to be no-slip walls.

The streamwise velocity profile obtained from HAMISH is shown within the simulation domain in Fig. 8a. The initial and final meshes for HAMISH with AMR are shown in Fig. 8b. The mesh is selectively refined close to the wall where the velocity gradients assume large values. The mesh spacing increases as the magnitude of the velocity gradient drops in the wall-normal direction. In Fig. 9 the variations of normalised streamwise velocity $u^+ = u/u_\tau$ with normalised wall normal distance y^+ from HAMISH (line with diagonal crosses) are compared to the corresponding results obtained from SENG+ and Code_Saturne with 8,000 equidistant grid points without employing any mesh stretching to ensure $y^+ \leq 0.6$ adjacent to the wall. The analytical solution for laminar fully developed channel flow (i.e. $u^+ = u/u_\tau = Re_\tau[y/h - 0.5(y/h)^2]$) is also shown in Fig. 9, and it can be seen that all numerical simulations capture the analytical profile accurately. It can be seen from Fig. 9 that HAMISH with AMR is capable of refining the mesh in boundary layer flows. The bulk mean

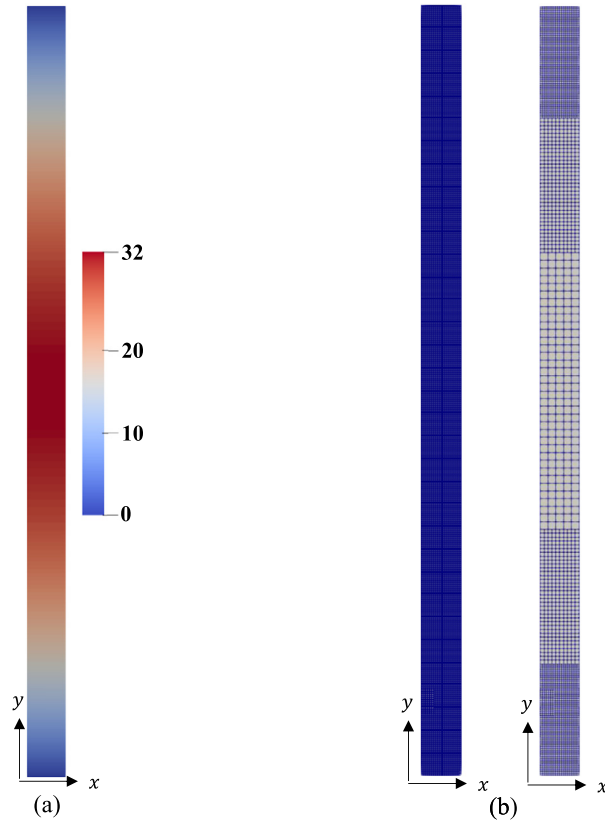


Fig. 8. (a) Normalised streamwise velocity $u^+ = u/u_\tau$ within the computational domain for $Re_\tau = 64$ in the case of fully developed channel flow simulation using HAMISH; (b) Initial (left) and final (right) meshes of the fully developed laminar channel flow simulation using HAMISH.

velocity $u_b = (1/2h) \int_0^{2h} u dy$ normalised by the friction velocity u_τ was obtained from HAMISH with and without AMR and takes the value 21.34 for both cases, while the analytical result is 21.33. The same quantity evaluated using SENG+ and Code_SATURNE is smaller by 0.05% and 0.025% respectively when compared with the values from HAMISH. The saving is CPU time when using HAMISH with AMR is 54.4% compared to HAMISH without AMR.

2-D expanding flame under quiescent laminar conditions

A 2-D expanding laminar flame propagating into a quiescent reactant premixture is simulated using HAMISH with a single step irreversible Arrhenius type chemical reaction with unity Lewis number (i.e. $Le = 1.0$), with parameters chosen to mimic stoichiometric methane-air combustion. A reaction progress variable c is defined according to $c = (Y_{R0} - Y_R)/(Y_{R0} - Y_{R\infty})$ where Y_R is the mass fraction of fuel (e.g. CH_4) and its value in the unburned reactants and fully burned products are denoted by Y_{R0} and $Y_{R\infty}$, respectively. The flame is initialised by specifying the steady state laminar flame solution in the radial direction such that the radius of the circular central region of burned products characterised by the value $c > 0.9$ is $r = 5.53\delta_{th}$ where δ_{th} is the thermal flame thickness. The 2-D simulation uses an initial mesh of 640×640 cells in a domain of $55.3\delta_{th} \times 55.3\delta_{th}$. All sides of the domain boundary have partially non-reflecting boundary conditions specified using the NSCBC technique [28,29]. The reaction progress variable field under the initial condition is shown in Fig. 10a and the initial mesh is shown in Fig. 11a. The AMR criteria for this case make use of the gradients of species and temperature. The evolution of the reaction progress variable field at two time instants $t = 0.03\delta_{th}/S_L$ and $0.15\delta_{th}/S_L$ are shown in Figs. 10b and 10c, respectively. The mesh evolution due to the moving flame front is shown in Figs. 11b and 11c. It is evident from Figs. 10b and 10c that the flame retains its circular shape, as expected in the laminar quiescent environment, while Figs. 11b and 11c indicate that the AMR leads to selective mesh refinement and de-refinement within the domain, especially across the flame where the gradients of species and temperature assume large values.

The normalised volume-integrated burning rate $\int_V \dot{w}_c dV / \rho_0 S_L A_0$, where $\dot{w}_c$ is the reaction rate of the reaction progress variable and A_0 is the initial area, was obtained from HAMISH with and without AMR and found to be equal to 3.461. The same quantity obtained from SENG+ with a resolution of 10 grid points within δ_{th} for the time instants reported in Figs. 10 and 11 was found to be 1% lower. The saving in CPU time when using HAMISH with AMR was found to be 64.8%.

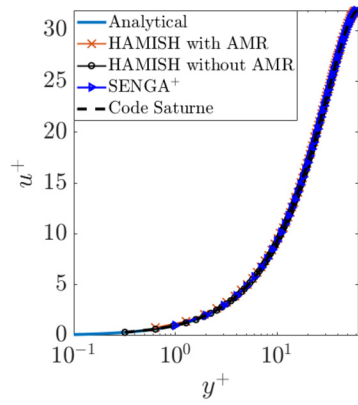


Fig. 9. Variation of normalised streamwise velocity u^+ with normalised wall normal distance y^+ as obtained from HAMISH, SENG+ and Code_Saturne for $Re_\tau = 64$.

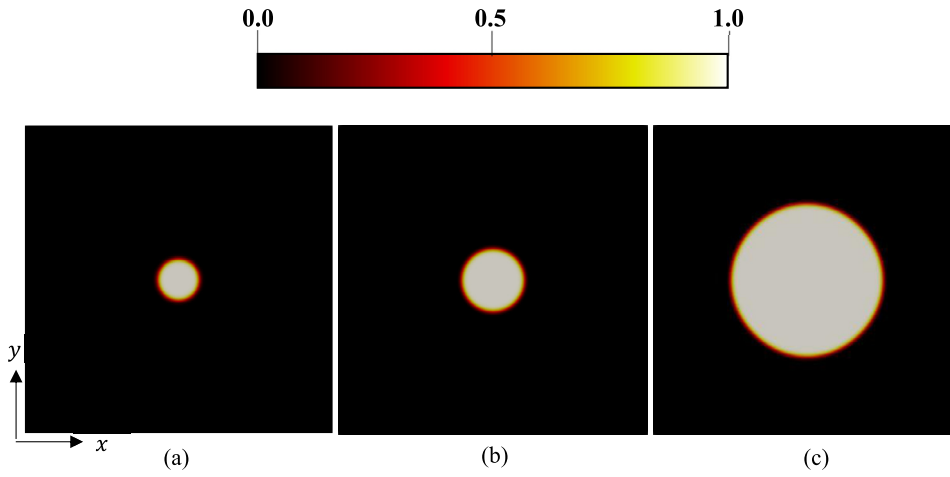


Fig. 10. The reaction progress variable c field evolution with time for the 2-D expanding ignition front under laminar quiescent condition at (a) $t = 0$, (b) $0.03\delta_{th}/S_L$ and (c) $0.15\delta_{th}/S_L$.

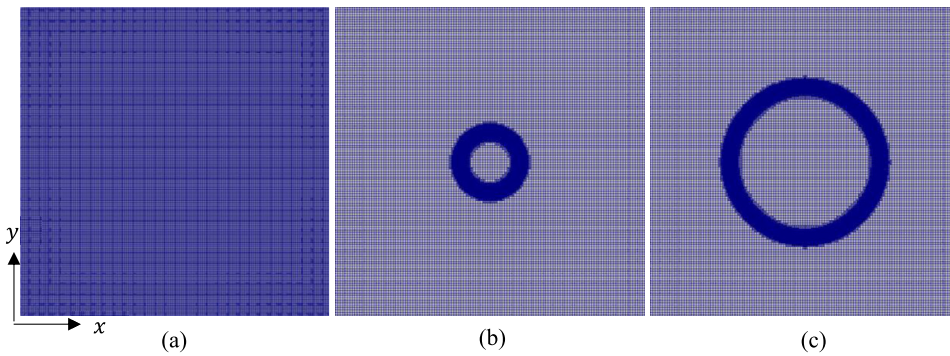


Fig. 11. The mesh evolution in time for the 2-D expanding ignition front under laminar quiescent condition at (a) $t = 0$, (b) $0.03\delta_{th}/S_L$ and (c) $0.15\delta_{th}/S_L$.

3-D Taylor-Green vortex simulation

The Taylor-Green vortex serves as a convenient problem for assessing accuracy and fidelity of high order DNS codes [39]. The Taylor-Green vortex test case involves DNS of a periodic transitional flow defined in terms of velocity components u , v and w and pressure p [39,40] by the initial condition:

$$u = V_0 \sin\left(\frac{x}{L}\right) \cos\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right); \quad v = -V_0 \cos\left(\frac{x}{L}\right) \sin\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right); \quad w = 0 \quad (15i)$$

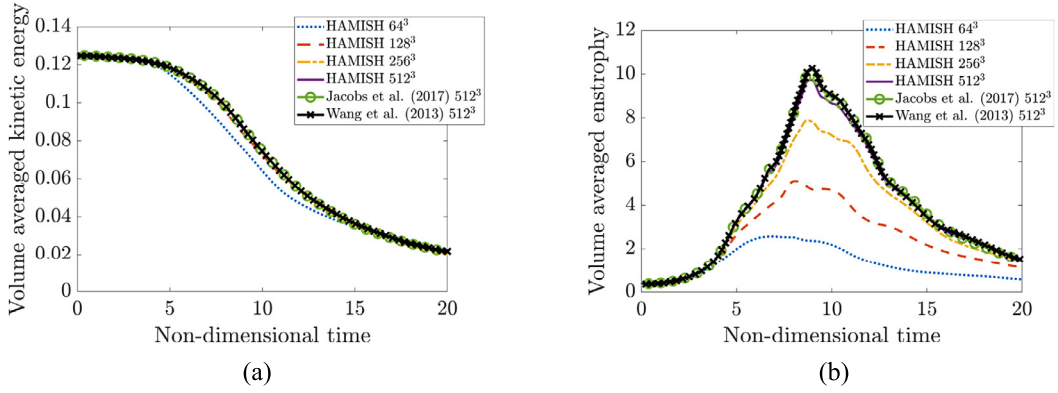


Fig. 12. Taylor-Green vortex simulation: Evolutions of (a) volume averaged kinetic energy, (b) volume averaged enstrophy with non-dimensional time (i.e. tV_0/L).

Table 2

Comparison of the HAMISH results for the Taylor-Green vortex case with the 512^3 grid with respect to the corresponding results obtained from codes using a spectral method [39] and a high-order difference scheme [40].

Maximum % difference	K	Ω
Results from [39]	-3.20%	-5.30%
Results from [40]	-2.31%	-4.49%

$$p = p_0 + \frac{\rho_0 V_0^2}{16} \left[\cos\left(\frac{2x}{L}\right) + \cos\left(\frac{2y}{L}\right) \right] \left[\cos\left(\frac{2z}{L}\right) + 2 \right] \quad (15ii)$$

where ρ_0 is the initial density, V_0 is a velocity scale and L is the wavelength for periodicity. This initial condition eventually gives rise to small scale turbulence which decays in a manner similar to homogeneous but anisotropic turbulence. For the present analysis, the Reynolds number $Re = \rho_0 V_0 L / \mu_0$ is taken to be 1,600 (i.e. $\rho_0 V_0 L / \mu_0 = 1,600$ where μ_0 is the initial dynamic viscosity) following several previous analyses [39,40]. The Mach number $Ma = V_0 / a_0$ (where $a_0 = \sqrt{\gamma p_0 / \rho_0}$ is the initial acoustic speed with γ being the ratio of specific heats at constant pressure and constant volume) is taken to be 0.1. Results for the temporal evolution of the volume-averaged turbulent kinetic energy $K = 1/2(\rho_0 V)^{-1} \int_V \rho u_i u_i dV$ and dissipation rate $\Omega = (\rho_0 V)^{-1} \int_V \mu (\partial u_i / \partial x_j) (\partial u_i / \partial x_j) dV$ are obtained from HAMISH using a series of uniform meshes with 64^3 , 128^3 , 256^3 and 512^3 cells for a domain given by $-\pi L < \{x, y, z\} < \pi L$ and are shown in Figs. 12a and 12b, respectively. The results obtained using HAMISH are compared to the corresponding results obtained from codes using a spectral method [39] and a high-order difference scheme [40] and good agreement is obtained for the 512^3 mesh. These findings indicate that HAMISH can be used for turbulent fluid flow simulations with accuracy that is comparable with other high-order codes at similar mesh resolution.

The maximum differences in the values of K and Ω between the HAMISH results for the 512^3 mesh and those obtained from codes using a spectral method [39] and a high-order difference scheme [40] are reported in Table 2. The findings from Fig. 12 and Table 2 reveal that HAMISH is able to offer sufficient accuracy for turbulent flow simulations.

3-D DNS of premixed flame-turbulence interaction under decaying turbulence

The flame-turbulence interaction for a statistically planar turbulent premixed flame under decaying turbulence has been simulated using HAMISH to demonstrate its capability in dealing with DNS of turbulent reacting flows. A statistically planar turbulent stoichiometric methane-air premixed flame with initial turbulence intensity of $u'/S_L = 5.0$ and normalised integral length scale of $l/\delta_{th} = 3.0$ has been considered. A single step Arrhenius-type chemical reaction mechanism representing stoichiometric methane-air combustion has been used since the role played by intermediate chemical species is not of interest for the present purpose. The initial turbulent velocity field is generated by applying diffusion to a field of initially uncorrelated uniformly distributed random numbers [41]. The temperature, density and species fields are initialised using a steady-state laminar flame solution and the simulation has been carried out for two initial eddy turn over times (i.e. $t_{sim} = 2l/u'$), which amounts to $1.2\delta_{th}/S_L$, and satisfies the criterion $t_{sim} = \max(\delta_{th}/S_L, l/u')$ for decaying turbulence simulations. The volume-integrated turbulent kinetic energy in the unburned gas ahead of the flame and the volume-integrated reaction rate did not change appreciably with time when the simulation is stopped. The temporal variations of volume-integrated kinetic energy and volume-integrated reaction rate are qualitatively similar to those shown in previous studies [42] and are not shown here for the sake of conciseness.

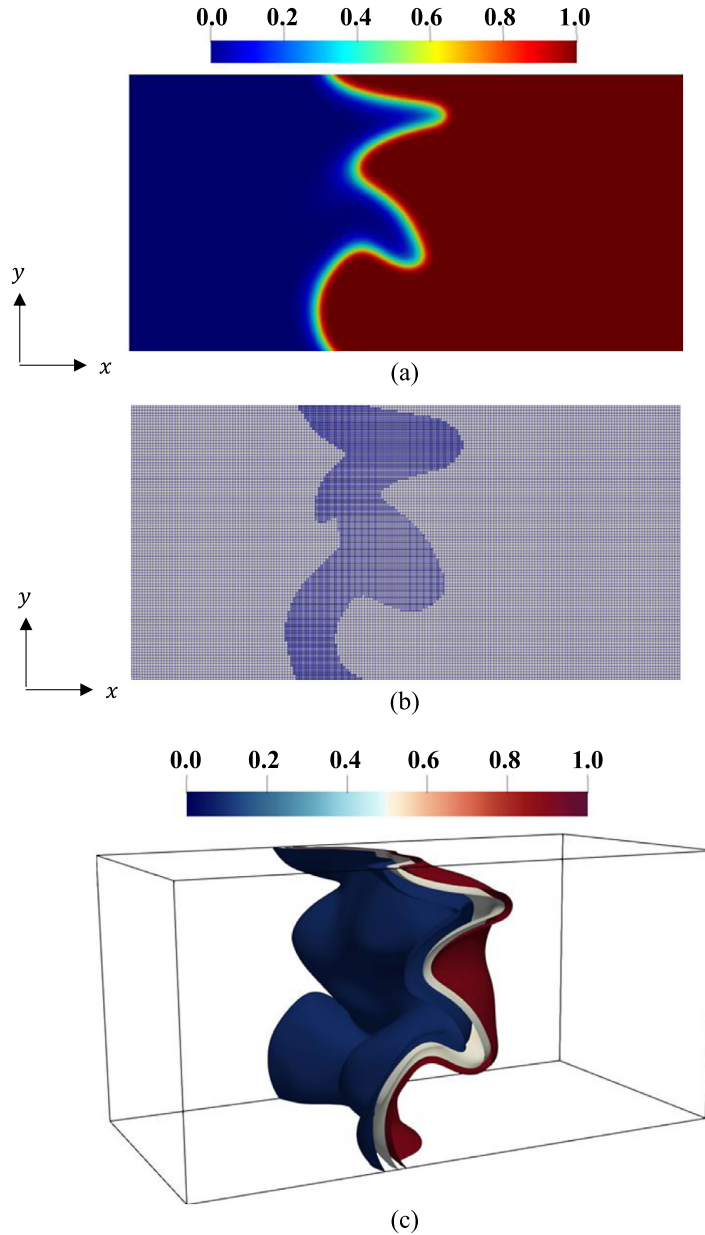


Fig. 13. The field of reaction progress variable c for the 3-D turbulent planar flame (a) contours of the reaction progress variable on the x - y mid plane; (b) mesh on the x - y mid plane; (c) 3-D isosurfaces of the reaction progress variable at $c = 0.1$, $c = 0.5$ and $c = 0.9$ at $t_{sim} = 2.0l/u'$.

An initial uniform Cartesian mesh of size $400 \times 200 \times 200$ (16 million cells) is used for a domain size of $40\delta_{th} \times 20\delta_{th} \times 20\delta_{th}$, which ensures 10 grid points within the thermal flame thickness δ_{th} . The contours of reaction progress variable c in the central mid-plane from the HAMISH simulation are shown in Fig. 13a and the corresponding AMR mesh with refinement criterion based on temperature gradient is shown in Fig. 13b. Corresponding instantaneous views of the isosurfaces of $c = 0.1, 0.5$ and 0.9 are shown in Fig. 13c. It is evident from Figs. 13a and 13b that AMR leads to selective refinement of the mesh across the flame where the gradients of species and temperature assume large values. It is also evident that de-refinement has taken place in the regions ahead of and behind the flame where only the Kolmogorov length scale needs to be resolved for the purpose of conducting a DNS of the non-reacting turbulent flow. The simulation finished with a total of 4 million cells, representing a four-fold reduction in the mesh required for this simulation as a consequence of using AMR. Comparisons for CPU time between AMR and static grids are not presented for this case because a static grid corresponding to the finest AMR mesh size would have required 81 billion grid points. Such a large simulation has not been carried out to date and is not feasible within currently available computational resources. This further emphasises the usefulness of AMR in production-level turbulent reacting flow simulations. In this case the results (i.e. turbulent burning velocity and flame

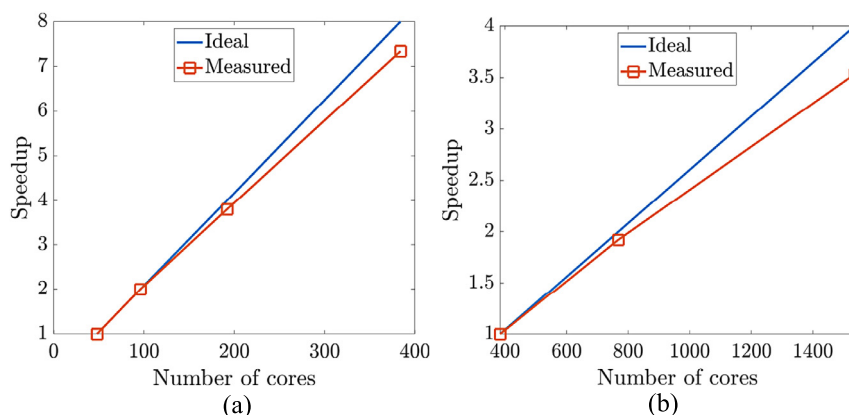


Fig. 14. Speed-up of HAMISH plotted against the ideal speed-up up to (a) 384 cores and (b) 1,536 cores.

surface area) obtained from HAMISH and SENG+ (with a grid size of $400 \times 200 \times 200$) remain in excellent agreement (i.e. the difference is $<1\%$ throughout).

Scalability analysis

A scaling analysis for HAMISH has been performed for a 3-D planar laminar methane-air premixed flame simulation with a skeletal chemical reaction mechanism involving 16 species and 25 reactions [31]. An equally-spaced Cartesian mesh was used, and the analysis was run on the UK national supercomputer ARCHER [38]. A mesh of $512 \times 128 \times 128$ cells was used on 48, 96, 192 and 384 cores, followed by a mesh of $1,024 \times 256 \times 256$ on 384, 768 and 1,536 cores. The scaling results are summarised in Fig. 14 where the parallel speed-up (i.e. runtime normalised with respect to the runtime on the smallest number of nodes) is plotted against the number of cores for each mesh. It can be seen from Fig. 14 that a parallel efficiency of 85.6% is obtained.

4. Conclusions

In this paper, the numerical and computational basis of the reacting-flow DNS code HAMISH has been described. The data structures and algorithms supporting AMR have been outlined together with the high-order spatial discretisation and Runge-Kutta time-marching schemes. The key capabilities of the HAMISH code have been demonstrated based on a number of test cases. These include (a) simulation of 1-D planar laminar premixed flames to demonstrate the capability of AMR in capturing the sharp gradients within the flame; (b) fully developed laminar channel flow, which shows that the code is capable of refining the mesh in the boundary layer next to the wall; (c) 2-D expanding flame under quiescent laminar condition, which demonstrates that the code is capable of dynamic local mesh refinement based on the position of the flame front; (d) 3-D non-reacting Taylor-Green vortex which demonstrates HAMISH can deal with vortical motion typical of turbulent flows; (e) 3-D premixed turbulent flame propagation under isotropic homogeneous decaying turbulence to demonstrate that the code can be employed for production level DNS of turbulent reacting flows. A parallel performance analysis of HAMISH based on a realistic 3-D turbulent reacting flow problem reveals 85.6% parallel efficiency up to 1,536 cores and indicates that the code may be used for production level simulations with large-scale parallelisation.

CRediT authorship contribution statement

R.S. Cant: corresponding author, conceptualisation, methodology, software, writing – original draft preparation, reviewing and editing.

U. Ahmed: software, testing, data generation, writing – original draft preparation, reviewing and editing.

J. Fang: software, testing, graphics, writing – reviewing and editing.

N. Chakarborty: supervision, writing - original draft preparation, reviewing and editing.

G. Nivarti: methodology, software, testing, writing – reviewing and editing.

C. Moulinec: supervision, methodology, writing – reviewing and editing.

D.R. Emerson: supervision, writing – reviewing and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

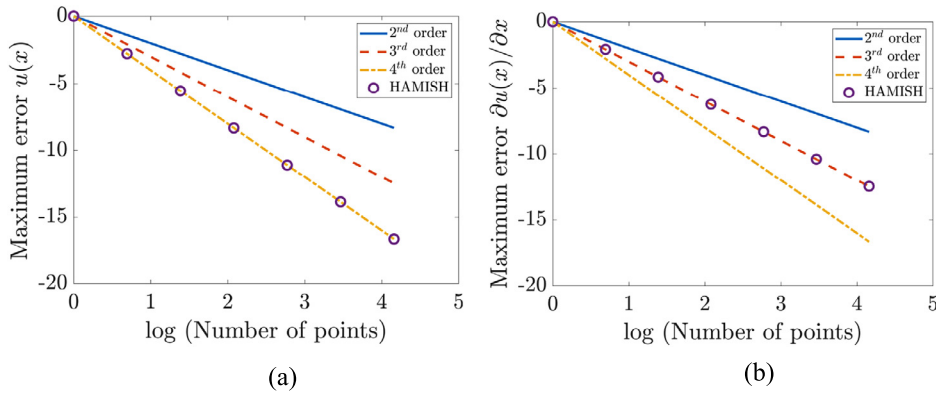


Fig. 15. Order of accuracy of the reconstructed function $u(x) = \sin(2\pi x/L)$ (a) and the order of accuracy of the first derivative of $u(x)$ (b). The lines with o symbol denote the results obtained from HAMISH.

Acknowledgements

The authors are grateful to EPSRC (EP/K025163/1, EP/P022286/1, EP/R029369/1) and ARCHER for financial and computational support. RSC is grateful to Prof. Bill Dawes for demonstrating the benefits of cubic cells in mesh generation [8], and to Dr Simon Harvey for pointing out reference [18].

Appendix A

For the assessment of the order of accuracy of the polynomial reconstruction, the velocity in the x -direction is specified to be a sinusoidal function $u(x) = \sin(2\pi x/L)$ for a 1-D periodic domain of length L equal to 1.0 cm. A fourth-order polynomial is used for the reconstruction as given by eq. (3) for evaluating the velocity u and its derivative (i.e. du/dx) at the cell faces (where the error is likely to be largest). The error relative to the analytical local value was calculated at each point, and the procedure was repeated for 32, 64, 128, 256, 512, 1,024 and 2,048 points in the x -direction. The maximum error in evaluating $u(x)$ and du/dx plotted against the logarithm of the number of points, both normalised to the values at 32 points, are presented in Figs. 15a and 15b, respectively. The results indicate that the polynomial reconstruction of the variable u is indeed 4th order accurate, and accordingly its derivative du/dx is indeed 3rd order accurate.

References

- [1] R.S. Cant, E. Mastorakos, *An Introduction to Turbulent Reacting Flows*, 1st edition, Imperial College Press, London, 2008.
- [2] T. Poinsot, D. Veynante, *Theoretical and Numerical Combustion*, R.T. Edwards Inc., Philadelphia, USA, 2001.
- [3] S.R. Turns, *An Introduction to Combustion: Concepts and Applications*, 3rd edition, McGraw-Hill, Hong Kong, 2011.
- [4] N. Chakraborty, R.S. Cant, E. Mastorakos, Effects of turbulence on spark ignition in inhomogeneous mixtures: a direct numerical simulation (DNS) study, *Combust. Sci. Technol.* 179 (1–2) (2007) 293–317.
- [5] R.K. Truelove, R.I. Klein, C.F. McKee, J.H. Holliman, L.H. Howell, J.A. Greenough, D.T. Woods, *Astrophys. J.* 495 (2) (1998) 821–852.
- [6] C. Wang, X. Dong, C.-W. Shu, Parallel adaptive mesh refinement method based on WENO finite difference scheme for the simulation of multi-dimensional detonation, *J. Comput. Phys.* 298 (2015) 161–175.
- [7] E.J. Kansa, Shock computations with adaptive mesh refinement and moving grids, *Comput. Math. Appl.* 15 (6–8) (1988) 623–634.
- [8] W.N. Dawes, Building blocks towards VR-based flow sculpting, *AIAA paper*, AIAA-2005-1156, 2005.
- [9] <https://ccse.lbl.gov/BoxLib/>.
- [10] P. Colella, D. Graves, T. Ligocki, D. Martin, D. Modiano, D. Serafini, B. Straalen, Chombo Software Package for AMR Applications - Design Document, Applied Numerical Algorithms Group, Computational Research Division, Lawrence Berkeley National Laboratory, Berkeley, CA, 2011.
- [11] P. MacNeice, K.M. Olson, C. Mobarry, R. de Fainchtein, C. Packer, A parallel adaptive mesh refinement community toolkit, *Comput. Phys. Commun.* 126 (3) (2000) 330–354.
- [12] J.B. Bell, AMR for low Mach number reacting flows, in: LBNL Report LBNL-54351, Proceedings of the Chicago Workshop on Adaptive Mesh Refinement Methods, 2003.
- [13] D. Fuster, G. Agbaglah, C. Josserand, S. Popinet, S. Zaleski, Numerical simulation of droplets, bubbles and waves: state of the art, *Fluid Dyn. Res.* 4 (2009) 065001.
- [14] G.M. Morton, A computer oriented geodetic data base and a new technique in file sequencing, *IBM Tech. Report*, 1966.
- [15] M. Bern, D. Eppstein, S.-H. Teng, Parallel construction of quadrees and quality triangulations, *Int. J. Comput. Geom. Appl.* 9 (6) (1999) 517–532.
- [16] K. Devine, B. Hendrickson, Tinkertoy parallel programming: a case study with Zoltan, *Int. J. Comput. Sci. Eng.* 1 (2005) 64.
- [17] Bloomberg, <http://leptonica.net/papers/colorquant.pdf>.
- [18] T. Tu, H. Yuy, L. Ramirez-Guzman, J. Bielak, O. Ghattas, K.-L. Mak, D.R. O'Hallaron, From mesh generation to scientific visualization: an end-to-end approach to parallel supercomputing, in: *Supercomputing 2006*, Tampa, Florida, USA, IEEE, 2006.
- [19] C. Burstedde, O. Ghattas, M. Gurnis, T. Isaac, G. Stadler, T. Warburton, L. Wilcox, Extreme-scale AMR, in: *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, November 13–19, 2010, pp. 1–12.
- [20] J.E. Flaherty, R.M. Loy, M.S. Shephard, B.K. Szymanski, J.D. Teresco, L.H. Ziantz, *J. Parallel Distrib. Comput.* 47 (1997) 139–152.
- [21] C.D. Norton, J.Z. Lou, T.A. Cwik, Status and directions for the PYRAMID parallel unstructured AMR library, 120, in: *Proc. of the 15th IEEE Int. Parallel and Distributed Processing Symp.*, IPDPS, 2001.

- [22] O. Lawlor, S. Chakravorty, T. Wilmarth, N. Choudhury, I. Dooley, Parfum: a parallel framework for unstructured meshes for scalable dynamic physics applications, *Eng. Comput.* 22 (3) (2006) 215–235.
- [23] T. Pringuey, R.S. Cant, High-order schemes on three-dimensional general polyhedral meshes – application to the level set method, *Commun. Comput. Phys.* 12 (2012) 1–41.
- [24] L. Ivan, C.P.T. Groth, High-order solution-adaptive central essentially non-oscillatory (CENO) method for viscous flows, *J. Comput. Phys.* 257 (2014) 830–862.
- [25] C.-W. Shu, S. Osher, Efficient implementation of essentially non-oscillatory shock-capturing schemes, *J. Comput. Phys.* 77 (1988) 439–471.
- [26] P. McCorquodale, P. Colella, A high-order finite-volume method for conservation laws on locally-refined grids, *Commun. Appl. Math. Comput. Sci.* 6 (2011) 1–26.
- [27] J.J. Gottlieb, C.P.T. Groth, Assessment of Riemann solvers for unsteady one-dimensional inviscid flows of perfect gases, *J. Comput. Phys.* 78 (1988) 437–458.
- [28] T. Poinsot, S.K. Lele, Boundary conditions for direct simulation of compressible viscous flows, *J. Comput. Phys.* 101 (1992) 104–129.
- [29] G. Lodato, P. Domingo, L. Vervisch, Three-dimensional boundary conditions for direct and large-eddy simulations of compressible viscous flows, *J. Comput. Phys.* 227 (2008) 5105–5143.
- [30] C.A. Kennedy, M.H. Carpenter, R.M. Lewis, Low-storage, explicit Runge–Kutta schemes for the compressible Navier–Stokes equations, *Appl. Numer. Math.* 35 (2000) 177–219.
- [31] M.D. Smooke, V. Giovangigli, Premixed and nonpremixed test flame results, in: *Reduced Kinetic Mechanisms and Asymptotic Approximations for Methane-Air Flames*, Springer, Berlin, Heidelberg, 1991, pp. 29–47.
- [32] K.W. Jenkins, R.S. Cant, Direct numerical simulation of turbulent flame kernel, in: *Recent Advances in DNS and LES*, Springer, 1999, pp. 191–202.
- [33] R.S. Cant, SENGAS User Guide, Tech. Report CUED-THERMO-2012/04, Cambridge University Engineering Department, 2012.
- [34] D.G. Goodwin, H.K. Moffat, R.L. Steph, Cantera: an object-oriented software toolkit for chemical kinetics, thermodynamics, and transport processes, <http://www.cantera.org>, 2018.
- [35] G.P. Smith, D.M. Golden, M. Frenklach, N.W. Moriarty, B. Eiteneer, M. Goldenberg, C.T. Bowman, R.K. Hanson, S. Song, W.C. Gardiner, J.V.V. Lissianski, Z. Qin, GRI-MECH 3.0, <http://www.me.berkeley.edu/grimech/>.
- [36] J. Li, Z. Zhao, A. Kazakov, F.L. Dryer, An updated comprehensive kinetic model of hydrogen combustion, *Int. J. Chem. Kinet.* 36 (2004) 566–575.
- [37] F. Archambeau, N. Mechitova, M. Sakiz, A finite volume method for the computation of turbulent incompressible flows - industrial applications, *Int. J. Finite* 1 (2004) 1–62.
- [38] ARCHER supercomputer, UK, <https://www.archer.ac.uk/>.
- [39] Z.J. Wang, et al., High-order CFD methods: current status and perspective, *Int. J. Numer. Methods Fluids* 72 (2013) 811–845.
- [40] C.T. Jacobs, S.P. Jammy, N.D. Sandham, OpenSBLI: a framework for the automated derivation and parallelexecution of finite difference solvers on a range of computer architectures, *J. Comput. Sci.* 18 (2017) 12–33.
- [41] A. Kempf, M. Klein, J. Janicka, Efficient generation of initial- and inflow-conditions for transient turbulent flows in arbitrary geometries, *Flow Turbul. Combust.* 74 (2005) 67–84.
- [42] N. Chakraborty, R.S. Cant, Influence of Lewis number on curvature effects in turbulent premixed flame propagation in the thin reaction zones regime, *Phys. Fluids* 17 (2005) 105105.